

SQL Interview Questions & Answers

68 questions covering fundamentals through window functions, indexing, transactions, and EF Core/.NET-adjacent topics — tailored for a ~1 year .NET/Azure developer preparing for interviews.

Section	Questions
1. SQL Fundamentals	Q1–Q12
2. Keys, Constraints & Schema Design	Q13–Q20
3. Aggregation & Grouping	Q21–Q28
4. Subqueries & CTEs	Q29–Q36
5. Window Functions	Q37–Q44
6. Joins & Set Operations, Applied	Q45–Q50
7. Indexes & Performance	Q51–Q56
8. Transactions, Locking & Security	Q57–Q63
9. .NET / EF Core-Adjacent	Q64–Q68

1. SQL Fundamentals

Q1. What is the difference between *WHERE* and *HAVING*?

WHERE filters individual rows before any grouping happens, and it cannot reference aggregate functions like *SUM()* or *COUNT()*. *HAVING* filters groups after *GROUP BY* has been applied, and it is used specifically to filter on aggregate results.

```
SELECT DeptId, COUNT(*) AS EmpCount
FROM Employees
WHERE IsActive = 1
GROUP BY DeptId
HAVING COUNT(*) > 5;
```

Q2. Explain *INNER JOIN* vs *LEFT JOIN* vs *RIGHT JOIN* vs *FULL OUTER JOIN*.

INNER JOIN returns only rows that have matching values in both tables.

LEFT JOIN returns all rows from the left table, plus matched rows from the right table; unmatched right-side columns are *NULL*.

RIGHT JOIN is the mirror of *LEFT JOIN* — all rows from the right table, matched or not.

FULL OUTER JOIN returns all rows from both tables, with *NULLs* wherever there's no match on either side.

```
SELECT e.Name, d.DeptName
FROM Employees e
LEFT JOIN Departments d ON e.DeptId = d.DeptId;
```

Q3. What is a *CROSS JOIN*? Give a real use case.

A *CROSS JOIN* produces the Cartesian product of two tables — every row from table A combined with every row from table B. A real use case is generating all possible combinations, such as pairing every product with every size/color variant, or building a calendar table crossed with a list of stores to get one row per store per day.

Q4. Difference between *DELETE*, *TRUNCATE*, and *DROP*.

DELETE removes rows one at a time, is logged, can be filtered with *WHERE*, and can be rolled back within a transaction.

TRUNCATE removes all rows at once, is minimally logged, resets identity columns, and is much faster — but you cannot filter which rows to remove.

DROP removes the entire table structure along with its data, indexes, and constraints — the table no longer exists after this.

Q5. UNION vs UNION ALL — performance implications.

UNION combines result sets and removes duplicate rows, which requires an internal sort/distinct operation and is therefore slower. UNION ALL simply concatenates result sets without checking for duplicates, so it is faster. Use UNION ALL whenever you know there won't be duplicates or you don't care about them.

Q6. Primary key vs foreign key vs unique key vs candidate key vs composite key.

A primary key uniquely identifies each row and cannot be NULL; a table can have only one.

A foreign key is a column that references the primary key of another table, enforcing referential integrity.

A unique key also enforces uniqueness but allows one NULL value (in most databases) and a table can have several.

A candidate key is any column (or set of columns) that could qualify as the primary key; one of the candidate keys is chosen to be the primary key.

A composite key is a primary key made up of two or more columns together.

Q7. VARCHAR vs NVARCHAR vs CHAR — storage and use cases.

CHAR is fixed-length and pads with spaces — good for values that are always the same length, like a 2-letter state code.

VARCHAR is variable-length, storing only the actual characters plus a small overhead — good for names, descriptions, etc.

NVARCHAR is the Unicode version of VARCHAR, using 2 bytes per character instead of 1, so it can store any language's characters at roughly double the storage cost. Use NVARCHAR whenever you need to support non-English text.

Q8. How does NULL behave in comparisons, aggregates, and joins?

NULL represents 'unknown', so any direct comparison to NULL (e.g. col = NULL) returns UNKNOWN, not TRUE — you must use IS NULL / IS NOT NULL. In aggregate functions, NULLs are ignored by SUM, AVG, MIN, MAX, and COUNT(column) — but COUNT(*) counts all rows regardless of NULLs. In joins, a NULL on the join key will never match another NULL, so those rows are excluded from an INNER JOIN result.

Q9. ISNULL() vs COALESCE() — key differences.

ISNULL() is SQL Server-specific, takes exactly two arguments, and returns a data type based on the first argument. COALESCE() is ANSI standard (portable across databases), accepts two or more arguments, and returns the first non-NULL value, with its data type determined by standard data type precedence rules. COALESCE is generally preferred for portability and flexibility.

Q10. What is a self-join? Give a scenario.

A self-join is a table joined to itself, typically to compare rows within the same table. A classic scenario is an Employees table with a ManagerId column that also references EmployeeId — you self-join to list each employee alongside their manager's name.

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.ManagerId = m.EmployeeId;
```

Q11. Difference between IN, BETWEEN, and LIKE.

IN checks if a value matches any value in a specified list. BETWEEN checks if a value falls within an inclusive range. LIKE performs pattern matching on strings using wildcards.

Q12. What are wildcard characters in LIKE, and how do you escape them?

% matches zero or more characters, and _ (underscore) matches exactly one character. To search for a literal % or _, you use the ESCAPE clause to define an escape character, for example LIKE '50%%' ESCAPE '%' to find a literal percent sign.

2. Keys, Constraints & Schema Design

Q13. What is referential integrity, and how do foreign keys enforce it?

Referential integrity means relationships between tables always stay consistent — you can't have a row referencing a parent row that doesn't exist. Foreign key constraints enforce this by preventing inserts/updates that reference a non-existent parent key, and by controlling what happens to child rows when a parent row is deleted or updated.

Q14. ON DELETE CASCADE vs ON DELETE SET NULL vs ON DELETE NO ACTION.

CASCADE automatically deletes child rows when the parent row is deleted.

SET NULL sets the foreign key column in child rows to NULL when the parent is deleted (the column must be nullable).

NO ACTION (the default) blocks the delete of a parent row if matching child rows still exist, raising an error instead.

Q15. Surrogate key vs natural key.

A natural key is a column that already exists in the real-world data and is unique, like an email address or SSN. A surrogate key is an artificial key with no business meaning, usually an auto-incrementing integer or a GUID, generated purely to identify rows. Surrogate keys are preferred because natural keys can change or turn out not to be as unique as assumed.

Q16. Explain 1NF, 2NF, 3NF, and BCNF with a short example each.

1NF (First Normal Form): each column holds atomic values and no repeating groups — e.g., don't store 'Phone1, Phone2' in one column; use a separate row or table instead.

2NF: must be in 1NF, and every non-key column must depend on the whole primary key, not just part of it. This matters for composite keys — e.g., in an OrderDetails table keyed on (OrderId, ProductId), a column like ProductName depends only on ProductId, so it shouldn't live there.

3NF: must be in 2NF, and no non-key column may depend on another non-key column (no transitive dependency). E.g., storing DeptName in the Employees table when it depends on DeptId, not on EmployeeId, violates 3NF.

BCNF (Boyce-Codd Normal Form): a stricter version of 3NF where every determinant must be a candidate key — it resolves rare edge cases 3NF doesn't cover, typically involving overlapping composite candidate keys.

Q17. What is denormalization, and when is it justified?

Denormalization intentionally introduces redundancy (e.g., storing a calculated total or duplicating a column) to reduce the number of joins needed and improve read performance. It's justified in read-heavy systems like reporting/data warehouses, where query speed matters more than avoiding update anomalies, and where the redundant data is refreshed through a controlled process.

Q18. Difference between a CHECK constraint and a DEFAULT constraint.

A CHECK constraint validates that data meets a specific condition before it's allowed into a column, e.g., CHECK (Salary > 0). A DEFAULT constraint supplies an automatic value for a column when no value is explicitly provided during an insert.

Q19. Can a table have multiple foreign keys? Multiple primary keys?

A table can have as many foreign keys as needed, each referencing a different (or the same) parent table. A table can only have one primary key — but that primary key can be composite, made up of multiple columns.

Q20. What is a computed column?

A computed column is a virtual column whose value is derived from an expression using other columns in the same table, rather than being stored directly (unless marked PERSISTED). For example, a TotalPrice column computed as Quantity * UnitPrice.

```
ALTER TABLE OrderDetails
ADD TotalPrice AS (Quantity * UnitPrice) PERSISTED;
```

3. Aggregation & Grouping

Q21. Difference between COUNT(*), COUNT(1), and COUNT(column_name).

COUNT(*) counts all rows, including those with NULLs, regardless of column values. COUNT(1) behaves identically to COUNT(*) in modern SQL Server — the optimizer treats them the same.

COUNT(column_name) counts only rows where that specific column is not NULL.

Q22. Why can't you use a column in SELECT without including it in GROUP BY (unless aggregated)?

Because GROUP BY collapses multiple rows into one row per group, SQL has no way to know which single value to display for a non-aggregated, non-grouped column when multiple different values exist within that group. Every column in the SELECT list must either be in the GROUP BY clause or wrapped in an aggregate function.

Q23. Write a query to find departments with more than 5 employees.

```
SELECT DeptId, COUNT(*) AS EmpCount
FROM Employees
GROUP BY DeptId
HAVING COUNT(*) > 5;
```

Q24. Filtering with WHERE before grouping vs HAVING after grouping.

WHERE filters rows before they're grouped, so it can exclude rows early and improve performance, but it can't reference aggregates. HAVING filters after grouping and aggregation, so it can use aggregate conditions. A well-optimized query often uses WHERE to reduce rows first, then HAVING only for aggregate-based conditions.

Q25. How do NULL values affect SUM(), AVG(), COUNT()?

All three aggregate functions ignore NULL values entirely rather than treating them as zero. This means AVG() divides by the count of non-NULL values, not the total row count, which can produce a different average than you'd expect if you assumed NULLs count as zero.

Q26. Write a query to get the count of employees per department, including departments with zero employees.

This requires a LEFT JOIN from Departments (not Employees), so departments with no matching employees still appear, with a count of 0.

```
SELECT d.DeptName, COUNT(e.EmployeeId) AS EmpCount
FROM Departments d
LEFT JOIN Employees e ON d.DeptId = e.DeptId
GROUP BY d.DeptName;
```

Q27. What is GROUPING SETS, ROLLUP, and CUBE?

These generate multiple levels of aggregation in a single query. ROLLUP produces a hierarchy of subtotals plus a grand total (e.g., by Region+City, then by Region, then overall). CUBE produces subtotals for every possible combination of the grouped columns. GROUPING SETS lets you explicitly specify exactly which grouping combinations you want, giving full control instead of the automatic hierarchy ROLLUP/CUBE produce.

```
SELECT Region, City, SUM(Sales) AS TotalSales
FROM SalesData
GROUP BY ROLLUP(Region, City);
```

Q28. Write a query to find the total and running total of sales per month.

```
SELECT MonthNo, SalesAmount,
SUM(SalesAmount) OVER (ORDER BY MonthNo
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal
FROM MonthlySales;
```

4. Subqueries & CTEs

Q29. What is a subquery? Scalar vs multi-row vs correlated subquery.

A subquery is a query nested inside another query. A scalar subquery returns exactly one value and can be used anywhere a single value is expected. A multi-row subquery returns multiple values and is typically used with IN, ANY, or ALL. A correlated subquery references a column from the outer query, so it runs once per outer row rather than once overall — making it conceptually similar to a loop.

```
-- Correlated subquery example
SELECT e.Name
FROM Employees e
WHERE Salary > (SELECT AVG(Salary) FROM Employees WHERE DeptId = e.DeptId);
```

Q30. EXISTS vs IN — which is generally more efficient and why?

EXISTS stops as soon as it finds one matching row (short-circuits), and it handles NULLs safely. IN materializes the full list of subquery results and compares each outer row against it — it can behave incorrectly if the subquery's list contains NULLs (with NOT IN especially). For correlated checks against large tables, EXISTS is generally more efficient and safer; for a short static list of literal values, IN is fine and often more readable.

Q31. What is a CTE, and how is it different from a temp table or view?

\

A CTE (Common Table Expression), defined with WITH, is a named temporary result set that exists only for the duration of the single statement that follows it — it's not stored anywhere and can't be indexed. A temp table (#Temp) is physically created in tempdb, persists for the session/connection, can be indexed, and can be reused across multiple statements. A view is a saved, reusable query definition stored permanently in the database and available to any query until it's dropped.

Q32. Write a recursive CTE to generate a number sequence from 1 to 10.

```
WITH Numbers AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM Numbers WHERE n < 10
)
SELECT n FROM Numbers
OPTION (MAXRECURSION 10);
```

Q33. Write a recursive CTE to display an employee-manager hierarchy.

```
WITH OrgChart AS (  
  SELECT EmployeeId, Name, ManagerId, 0 AS Level  
  FROM Employees  
  WHERE ManagerId IS NULL  
  UNION ALL  
  SELECT e.EmployeeId, e.Name, e.ManagerId, o.Level + 1  
  FROM Employees e  
  INNER JOIN OrgChart o ON e.ManagerId = o.EmployeeId  
)  
SELECT * FROM OrgChart ORDER BY Level;
```

Q34. When would you prefer a temp table over a CTE?

Prefer a temp table when you need to reuse the same intermediate result set across multiple queries or statements, when the result set is large and would benefit from its own index, or when you need to modify (update/delete) the intermediate data before using it further. CTEs are best for readability and one-off, single-statement use — especially recursion.

Q35. What is a derived table (inline view)?

A derived table is a subquery placed in the FROM clause and given an alias, effectively treated as a temporary, unnamed table for the scope of that query. It's similar to a CTE but defined inline rather than named up front with WITH.

```
SELECT dept_avg.DeptId, dept_avg.AvgSal  
FROM (SELECT DeptId, AVG(Salary) AS AvgSal FROM Employees GROUP BY DeptId) AS dept_avg  
WHERE dept_avg.AvgSal > 50000;
```

Q36. Difference between a CTE and a materialized/indexed view.

A CTE is re-evaluated every time the statement runs and stores nothing — it's just query text substitution. An indexed (materialized) view actually stores its result set physically on disk along with an index, so it's precomputed and can be read very quickly, at the cost of extra storage and the overhead of keeping it in sync when underlying data changes.

5. Window Functions

Q37. *ROW_NUMBER() vs RANK() vs DENSE_RANK() — with tied values.*

All three assign a sequential number to rows within a result set, ordered by a specified column, but they behave differently on ties. `ROW_NUMBER()` always gives a unique, sequential number even to tied rows (e.g., 1,2,3,4). `RANK()` gives tied rows the same rank but then skips the next rank number(s) (e.g., 1,2,2,4). `DENSE_RANK()` gives tied rows the same rank without any gap in the following numbers (e.g., 1,2,2,3).

Q38. *What does PARTITION BY do in a window function?*

`PARTITION BY` divides the result set into groups (partitions), and the window function is applied independently within each partition — similar to how `GROUP BY` creates groups, except `PARTITION BY` doesn't collapse rows; every original row is still returned, just with the calculated value attached.

Q39. *Write a query to find the top 3 highest-paid employees per department.*

```
SELECT * FROM (
  SELECT EmployeeId, Name, DeptId, Salary,
  DENSE_RANK() OVER (PARTITION BY DeptId ORDER BY Salary DESC) AS rnk
  FROM Employees
) ranked
WHERE rnk <= 3;
```

Q40. *What is LAG() and LEAD()? Give a use case.*

`LAG()` retrieves a value from a previous row in the result set, and `LEAD()` retrieves a value from a following row, both without needing a self-join. A common use case is comparing each month's sales to the previous month's sales to calculate month-over-month growth.

```
SELECT MonthNo, SalesAmount,
SalesAmount - LAG(SalesAmount) OVER (ORDER BY MonthNo) AS ChangeFromPrevMonth
FROM MonthlySales;
```

Q41. Write a query to calculate a running total of sales using `SUM()` `OVER()`.

```
SELECT OrderDate, SalesAmount,  
SUM(SalesAmount) OVER (ORDER BY OrderDate  
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal  
FROM Sales;
```

Q42. What's the difference between `NTILE()` and `RANK()`?

`RANK()` assigns a rank based on ordering, with gaps for ties. `NTILE(n)` instead divides the rows into `n` roughly equal-sized buckets/groups — for example `NTILE(4)` splits employees into quartiles by salary. `NTILE` is about grouping into a fixed number of buckets, not ranking individual rows.

Q43. Write a query to find the difference between each employee's salary and the department's average salary.

```
SELECT EmployeeId, Name, DeptId, Salary,  
Salary - AVG(Salary) OVER (PARTITION BY DeptId) AS DiffFromDeptAvg  
FROM Employees;
```

Q44. Can you use a window function directly in a `WHERE` clause? Why not, and how do you work around it?

No — window functions are logically evaluated after `WHERE` (and after `GROUP BY/HAVING`) in the query processing order, so their results aren't available yet when `WHERE` is applied. To filter on a window function's result, wrap the query in a subquery or CTE, compute the window function there, and then filter in the outer query's `WHERE` clause (as shown in Q39).

6. Joins & Set Operations, Applied

Q45. Write a query to find employees who have never placed an order.

```
SELECT e.EmployeeId, e.Name
FROM Employees e
LEFT JOIN Orders o ON e.EmployeeId = o.EmployeeId
WHERE o.OrderId IS NULL;
```

Q46. Write a query to find duplicate rows in a table based on specific columns.

```
SELECT Email, COUNT(*) AS Occurrences
FROM Customers
GROUP BY Email
HAVING COUNT(*) > 1;
```

Q47. Write a query to find employees earning more than their manager.

```
SELECT e.Name AS Employee, e.Salary, m.Name AS Manager, m.Salary AS ManagerSalary
FROM Employees e
INNER JOIN Employees m ON e.ManagerId = m.EmployeeId
WHERE e.Salary > m.Salary;
```

Q48. Write a query using *INTERSECT* and *EXCEPT* — explain what each returns.

INTERSECT returns only the rows that appear in both result sets. *EXCEPT* (called *MINUS* in some databases) returns rows from the first result set that do NOT appear in the second result set.

```
-- Customers who bought in both 2024 and 2025
SELECT CustomerId FROM Orders WHERE YEAR(OrderDate) = 2024
INTERSECT
SELECT CustomerId FROM Orders WHERE YEAR(OrderDate) = 2025;

-- Customers who bought in 2024 but NOT in 2025
SELECT CustomerId FROM Orders WHERE YEAR(OrderDate) = 2024
EXCEPT
SELECT CustomerId FROM Orders WHERE YEAR(OrderDate) = 2025;
```

Q49. Write a query to find the 2nd highest salary without using TOP/LIMIT.

```
SELECT MIN(Salary) AS SecondHighest
FROM (SELECT DISTINCT TOP 2 Salary FROM Employees ORDER BY Salary DESC) AS Top2;

-- Or, more scalable for the Nth highest:
SELECT Salary FROM (
SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
FROM Employees
) ranked
WHERE rnk = 2;
```

Q50. Write a query to pivot employee counts by department into columns.

This can be done with the PIVOT operator or with conditional aggregation (CASE + SUM), which is more portable across database engines.

```
-- Conditional aggregation approach
SELECT
SUM(CASE WHEN DeptName = 'Sales' THEN 1 ELSE 0 END) AS Sales,
SUM(CASE WHEN DeptName = 'IT' THEN 1 ELSE 0 END) AS IT,
SUM(CASE WHEN DeptName = 'HR' THEN 1 ELSE 0 END) AS HR
FROM Employees e
JOIN Departments d ON e.DeptId = d.DeptId;
```

7. Indexes & Performance

Q51. Clustered vs non-clustered index — how many of each can a table have?

A clustered index determines the physical storage order of the table's data — a table can have only one, since data can only be physically sorted one way. A non-clustered index is a separate structure that stores a sorted copy of key columns plus a pointer (or the clustering key) back to the actual row — a table can have many non-clustered indexes.

Q52. What is a covering index?

A covering index is a non-clustered index that includes all the columns needed to satisfy a query — the key columns plus any additional columns added via INCLUDE — so SQL Server can answer the query entirely from the index without having to look up the actual table data (no 'key lookup').

```
CREATE NONCLUSTERED INDEX IX_Employees_Dept
ON Employees (DeptId)
INCLUDE (Name, Salary);
```

Q53. How does an index affect INSERT/UPDATE/DELETE performance vs SELECT performance?

Indexes speed up SELECT queries by letting the engine seek directly to relevant rows instead of scanning the whole table. However, every INSERT, UPDATE, or DELETE must also update every index on the affected columns, so having too many indexes slows down write-heavy operations. It's a trade-off between read speed and write overhead.

Q54. What is index fragmentation, and how do you fix it?

Fragmentation happens when the logical order of index pages no longer matches their physical order on disk, often due to frequent inserts/updates/deletes, which forces extra I/O and hurts performance. It's fixed by rebuilding the index (ALTER INDEX ... REBUILD, which recreates it from scratch) for heavy fragmentation, or reorganizing it (ALTER INDEX ... REORGANIZE, a lighter online operation) for moderate fragmentation.

Q55. How would you read an execution plan to spot a missing index or a table scan?

You look for operators like 'Table Scan' or 'Clustered Index Scan' on large tables, which mean SQL Server is reading every row instead of seeking — a sign an index could help. You also look at the relative cost percentage of each operator to find the most expensive step, and SQL Server Management Studio will often suggest a 'Missing Index' directly in the graphical plan.

Q56. What is the difference between a seek and a scan?

A seek uses the index's B-tree structure to jump directly to the matching rows, which is fast and efficient for selective queries. A scan reads every row in the table or index sequentially, which is fine for retrieving large portions of data but expensive when only a few rows are actually needed.

8. Transactions, Locking & Security

Q57. What are the ACID properties?

Atomicity means a transaction either fully completes or fully rolls back — no partial changes. Consistency means a transaction takes the database from one valid state to another, respecting all rules and constraints. Isolation means concurrent transactions don't interfere with each other's intermediate states. Durability means once a transaction is committed, its changes survive even a system crash.

Q58. Explain transaction isolation levels.

Read Uncommitted allows dirty reads — you can see another transaction's uncommitted changes. Read Committed (SQL Server's default) only lets you see committed data, but values can change between reads within the same transaction (non-repeatable reads). Repeatable Read guarantees that once you've read a row, it won't change for the rest of your transaction, but new rows can still appear (phantom reads). Serializable is the strictest, fully isolating transactions from each other and preventing phantom reads, at the cost of concurrency. Snapshot isolation gives each transaction a consistent view of the data as of the transaction's start, using row versioning instead of locks.

Q59. What is a deadlock, and how do you diagnose/prevent one?

A deadlock occurs when two transactions each hold a lock the other one needs, so neither can proceed — SQL Server detects this and kills one transaction (the 'deadlock victim') to break the cycle. You diagnose deadlocks using SQL Server Profiler/Extended Events to capture deadlock graphs, and prevent them by always accessing tables in the same consistent order across all transactions, keeping transactions short, and using appropriate isolation levels.

Q60. Difference between optimistic and pessimistic concurrency.

Pessimistic concurrency assumes conflicts are likely, so it locks data as soon as it's read, blocking other transactions from modifying it until the lock is released. Optimistic concurrency assumes conflicts are rare, so it allows reads without locking and only checks for conflicts (e.g., via a rowversion/timestamp column) at the moment of update — if the data changed since it was read, the update fails and must be retried.

Q61. What is SQL injection, and how do parameterized queries prevent it?

SQL injection is an attack where malicious SQL code is inserted into a query through unsanitized user input, often by concatenating strings directly into a SQL statement — allowing an attacker to manipulate or extract data. Parameterized queries prevent this because the input values are passed separately from the SQL command text as typed parameters, so the database always treats them as literal data, never as executable SQL code.

```
// Safe (parameterized) example in ADO.NET
var cmd = new SqlCommand("SELECT * FROM Users WHERE Username = @username", conn);
cmd.Parameters.AddWithValue("@username", userInput);
```

Q62. Difference between a stored procedure and a user-defined function.

A stored procedure can perform actions with side effects (INSERT/UPDATE/DELETE), doesn't have to return a value, can return multiple result sets, and can't generally be used inside a SELECT statement. A function must return a value (scalar or table), cannot perform data modification through DML by design, and can be used directly within a SELECT statement or a JOIN.

Q63. What are triggers? Give a scenario for an AFTER INSERT trigger.

A trigger is special code that automatically runs in response to a DML event (INSERT, UPDATE, DELETE) on a table. An AFTER INSERT trigger is useful, for example, to automatically write an entry into an AuditLog table every time a new row is inserted into a sensitive table like Employees, capturing who/what/when without relying on the application code to do it.

9. .NET / EF Core-Adjacent

Q64. How does EF Core translate LINQ queries into SQL, and how do you inspect the generated SQL?

EF Core builds an expression tree from your LINQ query, then its query pipeline translates that tree into a provider-specific SQL command (e.g., T-SQL for SQL Server) at execution time. You can inspect the generated SQL by enabling logging (services.AddDbContext with `.LogTo(Console.WriteLine)`), using `ToQueryString()` on an `IQueryable`, or watching SQL Server Profiler / Extended Events.

Q65. What is the N+1 query problem, and how do `.Include()/projection` fix it?

The N+1 problem happens when you fetch a list of N parent entities, then separately query the database once per parent to get related child data — resulting in 1 + N round trips instead of one efficient query. `.Include()` fixes this by eager-loading related data in the same query (via a JOIN), and projecting only the needed fields with `Select()` avoids loading unnecessary related data altogether.

Q66. When would you use raw SQL (`FromSqlRaw`) or Dapper instead of EF Core?

You'd reach for raw SQL or Dapper when you need maximum performance for read-heavy, high-throughput queries where EF Core's tracking and translation overhead isn't worth it; when you need to call a complex stored procedure or use SQL features EF Core's LINQ provider doesn't translate well; or when working with an existing complex reporting query that's easier to hand-write than express in LINQ.

Q67. How do you handle database migrations in EF Core, and what can go wrong in a team setting?

Migrations are created with `Add-Migration` (or `dotnet ef migrations add`) and applied with `Update-Database` (or `dotnet ef database update`); each migration is a code file describing schema changes plus a snapshot of the model. In a team setting, conflicts arise when two developers create migrations against the same base model on different branches, leading to migration history conflicts or duplicate/incompatible model snapshots — this is usually resolved by rebasing carefully, regenerating migrations after merge, and never editing an already-applied migration that's shared with the team.

Q68. What's the difference between `AsNoTracking()` and normal EF Core queries?

By default, EF Core's change tracker keeps a snapshot of every entity it retrieves so it can detect changes for `SaveChanges()`. `AsNoTracking()` skips this bookkeeping, returning entities that EF Core won't monitor for changes — this reduces memory usage and improves query performance. Use it for read-only queries (e.g., displaying data) where you never intend to update the retrieved entities.